1 - Introduzione alle metriche e alle misurazioni software

Introduzione

L'importanza della misurazione

Se un qualcosa non si può migliorare, allora non si può migliorare. Di conseguenza, un qualcosa non misurabile non è neanche controllabile.

Definizioni

Si consideri un uomo la cui altezza è 1,81 metri. In questo caso, l'altezza rappresenta l'**attributo** che si desidera misurare. La **misurazione** è il processo che porta ad ottenere una **misura** (1,81) dell'attributo in termini della **metrica** scelta (il metro). Se si fosse scelto il centimetro come metrica, allora la misura sarebbe stata 181.

Formalmente:

- una **misura** è una rappresentazione numerica di una caratteristica o attributo di un prodotto o un processo software.
- una **metrica** è una scala o sistema di misurazione usata per quantificare una misura, ossia definisce la regola per l'assegnazione dei numeri alle misure
  - esempi: linee di codice (LOC), cyclomatic complexity (CC), defect density (DD)
  - è estremamente raro che una metrica rappresenti a pieno un concetto, pertanto, nella maggior parte dei casi, è necessario utilizzare più metriche per prendere una decisione.
- la **misurazione** è il processo di assegnazione dei numeri alle misure usando una metrica
  - può essere effettuata su una singola istanza o più istanze

Esempio

Si consideri l'asserzione "Progetto di ISPW = 5000 LOC".

- la misura è 5000
- la metrica è LOC
- la misurazione è il processo di conteggio delle LOC.

Scopo delle misurazioni

Una misurazione può essere motivata da diversi fattori:

- **valutazione**: è possibile valutare la qualità e l'efficienza di un processo di sviluppo software e di attività di manutenzione del codice
- **miglioramento**: è possibile identificare aree per effettuare miglioramenti e implementare cambiamenti che possono portare a migliori qualità, produttività ed efficienza del software
- **pianificazione**: si possono stimare le risorse necessarie a un progetto di sviluppo di software, quali tempo, effort e costo. Possono anche essere usate per pianificare e schedulare task e milestone del progetto
- **controllo**: si può monitorare e controllare il progresso di un progetto si sviluppo software, assicurandosi che sia in linea nei limiti del budget
- **comunicazione**: si possono comunicare lo stato e la qualità del progetto di sviluppo software agli stakeholder, quali project manager, clienti e altri membri del team di sviluppo.

Attributi di qualità delle metriche

Una buona metrica deve essere:

- **rilevante**: la metrica deve misurare un attributo importante del prodotto o processo software
- **obiettiva**: la metrica deve essere basata su dati verificabili e riproducibili (entro un certo errore), non soggetti a opinioni o stime
- **affidabile**: la metrica deve produrre risultati consistenti sotto diverse condizioni e da valutatori diversi
- **consistente**: la metrica deve essere consistente con gli obiettivi e il contesto del progetto software
- **facile da utilizzare**: la metrica deve essere facile da capire, calcolare e interpretare e non richiedere un effort o risorse eccessivi

Esempi di metriche

- **dimensione del software**: LOC, function points (FP), object points (OP)
- **qualità del software**: defect density (DD), fault frequency (FF), affidabilità
- **processo software**: produttività, efficienza, tempo di ciclo, tempo di esecuzione
- **complessità del software**: cyclometric complexity (CC), information flow metrics (IFM), Halstead's software science metrics
- **effort software**: costi, effort, schedulazione

Tipi di scale nelle misurazioni e l'analisi software

La misurazione e l'analisi del software include utilizzare metriche quantitative per valutare e migliorare la qualità, la produttività e l'efficienza del software. Al fine di effettuare misurazioni

sensate, occorre usare delle scale appropriate per riflettere la natura dei dati. In particolare, una **scala** è un attributo che caratterizza una metrica.

Di solito, la metrica si mappa 1:1 con la scala.

Esistono diversi tipi di scale:

- **nominale**
- **ordinale**
- **intervallo**
- **ratio**

Scale nominali

Una **scala nominale** è il tipo più semplice delle scale, in cui i valori sono assegnati a categorie o etichette senza alcun ordine o magnitudine. Le scale nominali rappresentano dati qualitativi, come i difetti software per tipo, come di UI, logica, prestazioni e sicurezza. Altri esempi sono i colori e i nomi.

Scale ordinali

Una **scala ordinale** è una scale in cui i valori sono assegnati a categorie o etichette in ordine, ma la magnitudine delle differenze tra i valori non è significante. Le scali ordinali rappresentano i dati che possono essere ordinati, ma non misurati, come i difetti software in base alla priorità (critica, alta, media e bassa).

Intervalli

Un **intervallo** è un tipo di scala in cui i valori sono assegnati a categorie o etichette in ordine, e la magnitudine della differenza tra i valori è significativa. Tuttavia, non esiste un vero "punto zero" sulla scala. Le scale a intervallo sono usate per rappresentare dati che possono essere misurati e confrontati, come l'effort nello sviluppo software in ore o giorni.

Ratio

La **ratio** è il tipo più sofisticato di scala, in cui i valori sono assegnati a categorie o etichette in ordine, la magnitudine della differenza tra i valori è significativa ed esiste un vero "zero point" sulla scala. Le scale ratio sono usate per rappresentare i dati che possono essere misurati e confrontati, come linee di codice, difetti per kLOC, o test coverage.

Statistiche per le scale

Le di una scala una scala sono incluse anche nelle scale più complesse.

Per capire il tipo si scala, si può ragionare sul tipo di operazioni che si possono effettuare su di esse.

Statistic / Scale type	Measure of: Central tendency	Dispersion	Dependency
Nominal	Mode	No., Frequency	
Ordinal	Median	Interval of variation	Spearman Correlation
Interval	Mean	Std. Deviation, Variance, Range	Pearson Correlation
Ratio	Geometric mean	Coefficient of variation	Statistical Tests

In generale, una scala più complessa consente di effettuare operazioni più complesse, quindi è più precisa.

Esempi

- capitoli di un libro -> ordinale
- colori -> nominale, ma dipende dal contesto (se si tratta dei colori delle cinture delle arti marziali, la scala è nominale)
- gate dell'aeroporto -> mista tra nominale (lettere) e ordinale (numeri)
- targa delle macchine -> intervallo (salvo eccezioni)
- numero di tratta dell'autobus -> come utente, nominale, ma per chi assegna i numeri è ordinale
- voti universitari -> intervallo
- voti alle elementari -> ordinali
- profitto -> ratio

Conclusione

Non c'è una scelta migliore nella scelta della scala: nelle misurazioni e nell'analisi software, essa dipende dalla natura dei dati misurati e l'uso che si intende fare dei risultati.

Capendo le differenze tra i tipi di scala, ci si può assicurare che le misurazioni siano appropriate, accurate e significative, portando a processi decisionali migliori e migliore qualità del software.

2 - Metriche software e loro utilizzo

ISO/IEC 25010:2011: System and software quality models

Standard ISO

ISO/IEC 25010:2011 è uno standard internazionale che fornisce un framework per valutare la qualità dei prodotti software. Lo standard è stato sviluppato dalla ISO (International Organization for Standardization) e dalla IEC (International Electrotechnical Commission).

Lo standard è progettato per essere usato da sviluppatori software, professionisti della quality assurance e altre agenti inclusi nel processo di sviluppo software.

Caratteristiche di qualità

- **funzionalità**: grado in cui il software incontra le necessità dei propri utenti.
- **affidabilità**: l'abilità del software di eseguire le funzionalità intese senza errori o failure (di solito misurata con il Mean Time to Failure, MTT)
- **usabilità**: facilità di utilizzo del software da parte degli utenti che vi interagiscono (misurata ad esempio con il numero di click necessari per eseguire una funzionalità)
- **efficienza**:
- **manutenibilità**: la facilità con cui il software può essere modificato, corretto o dattato ai requisiti in cambiamento
- **portabilità**:
- **compatibilità**:
- **sicurezza**: abilità del software di proteggere sé stesso e i suoi dati da accessi non autorizzati o attacchi.

Benefici

Lo standard ISO/IEC 25010:2011 fornisce diversi benefici alle organizzazioni e agli sviluppatori, tra cui:

- un approccio standardizzato per valutare la qualità del software consente obiettivi di confronto tra prodotti software
- un insieme comprensivo di metriche che possono essere usate per valutare i prodotti software tra più caratteristiche e sotto-caratteristiche di qualità
- qualità del software migliorata e rischio ridotto di difetti o vulnerabilità di sicurezza
- migliore comunicazione tra gli stakeholder coinvolti nel processo di sviluppo software
- soddisfazione e confidenza del cliente nel prodotto software aumentate

Metrice OO

Le **metriche OO** sono metriche usate per misurare la qualità dei sistemi costruiti con software OO. Aiutano gli sviluppatori a identificare falle nella progettazione, problemi di prestazioni e problemi di manutenibilità ancora prima nel ciclo di sviluppo software.

Ci sono diverse metriche OO, come le metriche CK, le quali possono essere usate per valutare la qualità di un sistema software object-oriented.

Metriche CK

Le **metriche CK**, anche dette metriche di Chidamber e Kemerer, sono state sviluppate da Shyam R. Chidamber e Chris F. Kemerer nel 1994.

Le metriche CK consistono di 6 diverse metriche che possono essere usate per misurare vari aspetti della progettazione software object-oriented, quali ereditarietà, coupling e complessità.

Elenco delle metriche

- **Weighted Methods per Class** (**WMC**) -> misura la complessità di una classe contando i metodi che contiene
- **Depth of Inheritance** Tree (**DIT**) -> misura la profondità della gerarchia di ereditarietà di una classe
- **Number of Children** (**NOC**) -> numero di sottoclassi dirette di una classe
- **Coupling Between Object classes** (**CBO**) -> numero delle classi con cui la cui la classe in questione è coupled (**Nota**: una classe usa altre classe o è usata, quindi in base a questo i cambiamenti possono essere più o meno impattanti)
- **Response For a Class** (**RFC**) -> misura il numero di metodi che possono essere invocati per rispondere a messaggi in un oggetto di una classe
- **Lack of Cohesion in Methods** (**LCOM**) -> misura la mancanza di coesione tra i metodi di una classe

Altre metriche OO

In aggiunta alle metriche CK, molte altre metriche object-oriented possono essere usate per misurare diversi aspetti della progettazione software. Alcune si queste metriche aggiuntive sono:

- **Lines of Code** (**LOC**)
- **Cyclomatic Complexity** (**CC**)
- **Fan-in** and e **Fan-out**
- **Class Responsibility Collaboration** (**CRC**)
- **Efferent Coupling** (**EC**)

- **Afferent Coupling** (**AC**)

Cyclomatic Complexity (CC)

La **cyclomatic complexity** misura la complessità di in programma in termini del numero di cammini indipendenti attraverso il codice. I programmi con un alto CC sono più difficili da capire e manutenere. La CC può anche essere calcolata utilizzando altri strumenti, come quelli di analisi statica del codice.

Fan-in e Fan-out

Il **fan-in** misura il numero di altre classi che dipendono da una data classe. Il **fan-out**, d'altra parte, misura il numero di classi da cui una certa classe dipende. Queste metriche possono essere usate per identificare classi che sono strettamente accoppiate ad altre classi e necessitano un refactoring.

Function point

I **function point** sono un insieme di tecniche di misurazione della dimensione del software che quantificano le funzionalità fornite da un'applicazione software. Essi sono indipendenti dal linguaggio di programmazione, dalla tecnologia e dai dettagli implementativi. Logicamente, misurano quante funzioni ha un'applicazione software, in particolare assegnando dei punti a ogni tipo di funzionalità. I FP sono basati sulla prospettiva dell'utente e dal valore di business dell'applicazione software.

Esiste anche un concetto di prossimità temporale che impatta l'affidabilità di una stima. In questo caso, si fa un compromesso tra accuratezza e tempo.

I FP tengono conto solo dei requisiti funzionali dell'applicazione e non di quelli non funzionali. **Problema**: questi ultimi impattano non poco su costi e tempo di sviluppo

Componenti dei FP

- **Internal Logical Files** (**ILFs**) -> dati mantenuti e usati dall'applicazione
- **External Interface Files** (**EIFs**) -> dati passati tra l'applicazione software e altri sistemi esterni
- **External Inputs** (**EIs**) -> input utente che avviano qualche processamento all'interno dell'applicazione software
- **External Outputs** (**EOs**) -> output utente risultanti da certo processamento all'interno dell'applicazione
- **External Inquiries** (**EQs**) -> indagini utente che risultano nella fornitura di informazioni da parte dell'applicazione

Calcolo de function point

Nei function point, ad ogni componente è assegnato un peso (basso, medio, alto) in base alla sua complessità e impatto sull'applicazione software. Le componenti pesate sono poi moltiplicate per un fattore di conversione per ottenere il numero totale di function point.

Il fattore di conversione varia a seconda della tecnologia di sviluppo software, della natura dell'applicazione software e il livello di dettaglio nei requisiti funzionali.

Pesi delle complessità dei function point:

Parametro di misurazione	Semplice	Medio	Alto
Numero di input utente	3	4	5
Numero di output utente	4	5	7
Numero di indagini utente	3	4	6
Numero di file	7	10	15
Numero di interfacce esterne	5	7	10

Parametro di misurazione	Conteggio	Fattore di peso (medio)	Totale
Numero di input utente	50	4	200
Numero di output utente	40	5	200
Numero di indagini utente	35	4	140
Numero di file	6	10	80
Numero di interfacce esterne	4	7	28

Ogni tipo di FP viene moltiplicato per un peso, dovuto alla complessità (simple, average, complex). Questo tipo di conto comporta dei problemi:

- sono stati assegnati dei valori a delle parole
- due simple valgono un complex
- valori fuori dal range devono essere comunque adeguati a questi tre valori.

Vantaggi

I FP forniscono uno standard e una misura consistente della dimensione software e della complessità. Sono basati sulla prospettiva dell'utente e il valore di business dell'applicazione software, non sulla tecnologia o sui dettagli implementativi. Possono essere usati per stimare gli sforzi di sviluppo software, il costo, la schedulazione, così come confrontare le applicazioni software tra diversi domini e piattaforme.

Limitazioni

I FP non catturano tutti i tipi di funzionalità software e attributi di qualità, come usabilità, manutenibilità e affidabilità. Infatti, sistemi funzionalmente simili ma orientati a un pubblico diverso, posso prevedere picchi di carico diversi e quindi requisiti non funzionali diversi.

Inoltre, possono essere soggetti a interpretazioni e giudizi soggettivi, dato che i pesi e i fattori di conversione possono variare tra i vari stakeholder e contesti.

Possono anche essere non adatti a tutti i tipi di applicazioni software, come a quelli con grande enfasi sull'interfaccia utente o processamento real-time.

Strategie GQM-S

È un approccio dell'ingegneria del software che mette insieme due metodi:

- **Goal-Question-Metric** (**GQM**), per definire gli obiettivi, le domande e le metriche
- **Strategies**, per identificare e selezionare le strategie più appropriate per raggiungere i dati obiettivi

Si inizia con i **business goal**, ossia obiettivi business. Si inizia da questi per, ad esempio, migliorare la soddisfazione del cliente, ridurre i costi e i tempi di sviluppo. A volte occorre fare feature implementation piuttosto che testing per uscire prima sul mercato e accaparrarsi l'utenza, aggiustando poi in seguito l'applicazione.
Le **strategie** definiscono i compromessi per identificare gli specifici obiettivi software che contribuiscono a raggiungere gli obiettivi di business.

I **software goal** puntano invece a migliorare l'efficacia dei test sul sistema. Questi sono raggiunti usando degli scenari per definire degli obiettivi di misurazione.

Infine, gli obiettivi di misurazione consentono di derivare delle misure per interpretare obiettivi di livello più alto.

Processo

Il processo è estremamente iterativo ed è diviso in fasi:

1. **Initialize**: si definiscono l'impegno, il personale e le responsabilità
2. **Characterize**: si definisce lo scopo dell'applicazione e si caratterizza l'ambiente e il contesto
3. **Set goals**: si determinano le strutture organizzative, si esegue un'analisi dello status quo, si pone una priorità sugli obiettivi e si costruisce una griglia di obiettivi/strategie
4. **Choose process**: si pianifica l'implementazione di strategie , si organizza la raccolta e l'analisi dei dati e si definiscono i meccanismi di feedback. Inoltre, il processo target può essere modificato in base alle necessità, se necessario, e si attua il piano di misurazione sul processo target
5. **Execution model**:

    1. si eseguono le strategie
    2. si raccolgono e si analizzano i dati
    3. si fornisce un feedback
6. **Analyze results**: si analizzano i dati e si rivedono le strategie, si rivedono e si comunicano i risultati e si analizzano i costi/benefici
7. **Package and improve**: si adattano e si migliorano gli obiettivi e le strategie, si rivedono i contesti e le assunzioni e si registrano le lezioni apprese.

Utilizzi

- miglioramento del processo di sviluppo software: un team di sviluppo software può usare GQM+S per identificare le area di miglioramento nel loro processo di sviluppo software. Possono impostare obiettivi e strategie per misurare e migliorare la qualità, l'efficienza e l'efficacia del loro processo di sviluppo software.
- valutazione della qualità del software: un'organizzazione di sviluppo software può usare GQM+S per verificare la qualità del loro prodotto software. Possono impostate obiettivi e strategie per misurare le caratteristiche di qualità dei loro prodotti software, quali affidabilità, usabilità e manutenibilità.
- gestione del software: un project manager può usare GQM+S per monitorare il progresso di un progetto di sviluppo software. Possono impostare gli obiettivi e le strategie per misurare il progresso del progetto, quali il tasso di completamento di milestone, la produttività dei membri del team e la soddisfazione degli stakeholder
- decision making: un'organizzazione di sviluppo software può usare GQM+S per supportare il decision making. Possono impostare gli obiettivi e le strategie per raccogliere e analizzare i dati per supportare il decision making, come decidere sull'adozione di una nuova tecnologia o selezionare una metodologia di sviluppo software.
- software testing: un team di software testing può usare GQM+S per migliorare il loro processo di testing. Possono impostare obiettivi e strategie per misurare l'efficacia del loro processo di testing, come la coverage dei test case, l'identificazione di difetti e il tempo richiesto per risolvere un difetto.
- manutenzione del software: un team di manutenzione del software può usare GQM+S per migliorare il processo di manutenibilità. Possono impostare gli obiettivi per misurare l'effort della manutenzione, come il tempo richiesto per risolvere un difetto, la frequenza dei cambiamenti e la soddisfazione degli utenti con gli aggiornamenti software.

Esempio 1

1.
2. Value -> tempo di esecuzione dei test ridotto da 10s a 8s
3. due punti
   - strategia -> aumentare la valutazione di 0,5 stelle
   - value -> rating passato da 4 a 4,5 stelle

La strategia non definisce esattamente "come" raggiungere l'obiettivo, ma parzialmente, aiutando poi a definirlo.

Esempio 2

1. 3
   - metrica -> tempo medio necessario per chiudere un ticket
   - strategia -> ridurre il tempo medio del 10%
   - valore -> riduzione del tempo da 10 giorni a 9 giorni
2. 3
   - metrica -> numero di ticket risolti per iterazione
   - strategia -> aumentare il numero di ticket del 10%
   - valore -> numero di ticket aumentato da 10 a 11

   In generale, lo scopo non è fare un sistema in cui si può barare. L'obiettivo è creare un sistema abbastanza accurato, ma al contempo facile da utilizzare.

3 - Introduzione alla stima dei costi

Definizioni

Una **stima** è un proxy per una misurazione, ossia viene utilizzata quando la misurazione non è possibile. Una **predizione** ha funzione simile, ma relativa al tempo. In breve, se un concetto è presente si parla di stima, se è futuro di predizione.

Domande fondamenti sulla stima

Quando si effettua una stima, occorre porsi diverse domande:

- quanto effort è richiesto per completare l'attività?
- quando tempo (in giorni) è necessario per completare l'attività?
- qual è il costo totale dell'attività?

Componenti del costo software

Nello sviluppo si deve tener conto dei seguenti costi:

- costi hardware e software
- costi di viaggio e di training
- costo dell'**effort**, fattore dominante nella maggior parte dei progetti
    - salari degli ingegneri coinvolti nel progetto
    - costi sociali e assicurativi

L'ultimo punto deve considerare anche le spese generali:

- costo di edifici, riscaldamento, illuminazione
- costo di networking e comunicazione
- costo di strutture condivise (come librerie, staff dei ristoranti ecc.)

Costo e prezzo

Le stime vengono effettuate per scoprire il costo dello sviluppatore affinché produca un sistema software. Non vi è una relazione semplice tra il costo di sviluppo e il prezzo richiesto al cliente. Infatti, quest'ultimo è influenzato da fattori economici, politici, aziendali e organizzativi a livello più alto.

Ruolo del linguaggio di programmazione

	Analysis	Design	Coding	Testing	Documentation
Assembly code	3 weeks	5 weeks	8 weeks	10 weeks	2 weeks
High-level language	3 weeks	5 weeks	8 weeks	6 weeks	2 weeks

	Size	Effort	Productivity
Assembly code	5000 lines	28 weeks	714 lines/month
High-level language	1500 lines	20 weeks	300 lines/month

Tecniche di stima

Si possono utilizzare diverse tecniche di stima:

- modellazione algoritmica del costo
- giudizio esperto
- stima per analogia
- legge di Parkinson
- prezzi vincenti

Modellazione algoritmica del costo

È un approccio **formulare** basato su informazioni storiche del costo e che generalmente si basa sulla dimensione del software (vedi COCOMO).

Giudizio esperto

Uno o più esperti sia nello sviluppo software, sia nel dominio applicativo, usano la loro esperienza per predire il costo del software. Il processo itera fino al raggiungimento di un consenso.
Vantaggi: il metodo di stima è relativamente economico e può essere accurato se gli esperti hanno esperienza diretta su sistemi simili.
Svantaggi: è molto inaccurato se non ci sono esperti. Un altro problema è capire chi è esperto o meno relativamente a un dominio applicativo, ossia definire un grado di competenza che pio induca un grado di variabilità sulla stima.

Stima per analogia

Il costo di un progetto è calcolato confrontando il progetto a un altro simile nello stesso dominio applicativo.
Vantaggi: è accurato se sono disponibili dati sul progetto
Svantaggi: è impossibile se non è disponibile alcun progetto simile; è richiesto infatti un database dei costi mantenuto sistematicamente.

Legge di Parkinson

Il progetto costa tanto quanto le risorse sono disponibili. La gestione del tempo è solo psicologica. Ci viene spontaneo ritmare i tempi per portare a termine un progetto in tempi

brevi. Lo stesso compito può richiedere un'ora o una settimana, a seconda del tempo che ci concediamo per portarlo a termine.
Vantaggi: nessuna spesa eccessiva
Svantaggi: il sistema di solito rimane incompiuto.

Prezzo vincente

Il progetto costo tanto quanto il cliente ha da spendere su di esso.
Vantaggi: si ottiene comunque il contratto
Svantaggi: la probabilità che il cliente ottenga il sistema desiderato è piccola; infatti, i costi non riflettono accuratamente il lavoro richiesto.

Introduzione al planning poker

Il **Planning Poker** è una tecnica di stima agile che è diventata molto diffusa negli ultimi anni.

Nonostante le basi delle tecnica sia stati in giro per molti anni, le rifiniture apportate da Greening lo rendono usabile dai team agile e lo hanno sfruttato a pieno.

Il Planning Poker è estremamente semplice da giocare, ma al contempo abbastanza accurato da usare per il planning agile.

Regole

1. Ogni partecipante ottiene un mazzo di carte di stima rappresentante una sequenza di numeri. Le sequenze più note richiedono il raddoppiamento di ogni carta (0, 1/2, 1, 2, 4, 8, 16 ecc.) oppure la sequenza di Fibonacci (1, 2, 3, 5, 8, 13 ecc.), ma quest'ultima è di solito più popolare. Il fatto di utilizzare un mazzo incompleto (senza tutti i numeri) serve a dare un concetto di limitatezza del range e di vicinanza tra i valori: l'approccio è sicuramente meno preciso, ma più facile da utilizzare (compromesso).
2. Il moderatore presenta una user story alla volta al team
3. Il Product Owner (o un ruolo equivalente) risponde a ogni domanda che il team può avere riguardo la story
4. Ogni partecipante seleziona *privatamente* una carta, rappresentante la sua stima sulla dimensione della user story. La seleziona privata evita il concetto di *anchoring*, ossia la tendenza ad essere d'accordo con gli altri ed pensare il meno possibile.
5. La size rappresenta un valore che tiene conto di tempo, rischio, complessità e altri fattori rilevanti
6. Quando tutti sono pronti con una stima, tutte le carte sono presentate simultaneamente
7. Se c'è consenso su un particolare numero, poi la dimensione è registrata e il team si muove alla story successiva
8. Nel (molto probabile) caso in cui la stima differisca, il più alto e il più basso stimatore difendono la loro stima contro il resto del team
9. Il gruppo dibatte brevemente gli argomenti

10. Viene fatto un nuovo round di stima (passi 4-9) (**Nota**: non si presenta anchoring, dato che tutti hanno espresso la propria opinione e si ha modo di valutarle tutte autonomamente)
11. Si continua finché non si raggiunge il consenso e il moderatore registra la stima
12. Si ripete per tutte le story

Dettagli

CI sono diverse buone ragioni per cui la maggior parte dei trainer e dei coach agile promuovono l'uso del Planning Poker:

- favorisce la **collaborazione** includendo l'intero team
- crea stime basate su **consenso** anziché avere una sola persona che deriva le stime
- espone i **problemi** prima tramite discussione di ogni user story.

Consigli

- **coloro che possono davvero fare il lavoro sono quelli che votano**: molto spesso i team agile fanno votare tutti, anche se non hanno idea riguardo il lavoro richiesto dalla storia
- **i manager non votano**: i manager sono di solito incentivati a volere che il lavoro richieda meno tempo, quindi spesso abbassano il voto di troppo
- quando c'è un **pareggio** nella votazione tra due dimensioni che sono consecutive, si prende **la più grande** e si va avanti
- **usare un timer o qualsiasi altra cosa per limitare la discussione**: si possono ad esempio limitare le discussioni a non più di un minuto
- **usare la carta "Ho bisogno di una pausa"**: troppo spesso i team staranno lavorando duro per giocare Planning Poker e non realizzano che alcuni hanno bisogno di una pausa
- **fare incontrare la persona che crea le user story con i lead di QA e Sviluppo prima del Planning Poker** per assicurarsi che le user story abbiano risposte ai quesiti più ovvi che un team può chiedere. Ciò consente al team di concentrarsi sulla dimensione, senza usare il proprio tempo per raccogliere informazioni perché la user story è scritta male
- **ricordare la baseline**: qualunque baseline il team sceglie, essa deve essere consistente da iterazione a iterazione. Se un giorno ideale è di dimensione 1, allora tutte le iterazioni devono usare quel punto di riferimento.

4 - Architetture software

Cos'è un'architettura software

Una definizione comunemente accettata di architettura software deriva dallo standard IEEE 1471:

> L'organizzazione fondamentale del sistema, incorporata nelle sue componenti, le relazioni tra loro e verso l'ambiente e i principii che governano la sua progettazione e la sua evoluzione.

Si può considerare anche una definizione dello scopo di u'architettura software:

> Il principale intento di un'architettura software è fornire un controllo intellettuale su un sistema sofisticato di enorme complessità.
> ~ Kruchten

Un'architettura software è sempre presente

Ogni sistema ha la sua architettura, che può essere:

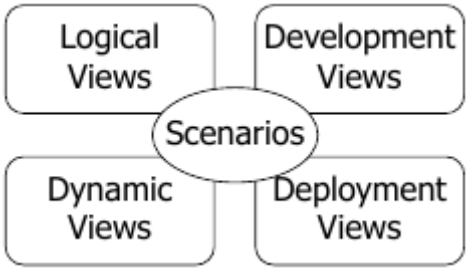
- **intenzionale** (pianificata, progettata) -> forward engineering
  - esplicita
  - documentata
- **non intenzionale** (spontanea, accidentale) -> reverse engineering
  - implicita
  - da ricostruire
  - da documentare

Principi fondamentali

> La separazione di competenze e viste (4+1) spezza la complessità del sistema, consentendone una migliore gestione.
> ~ Kruchten

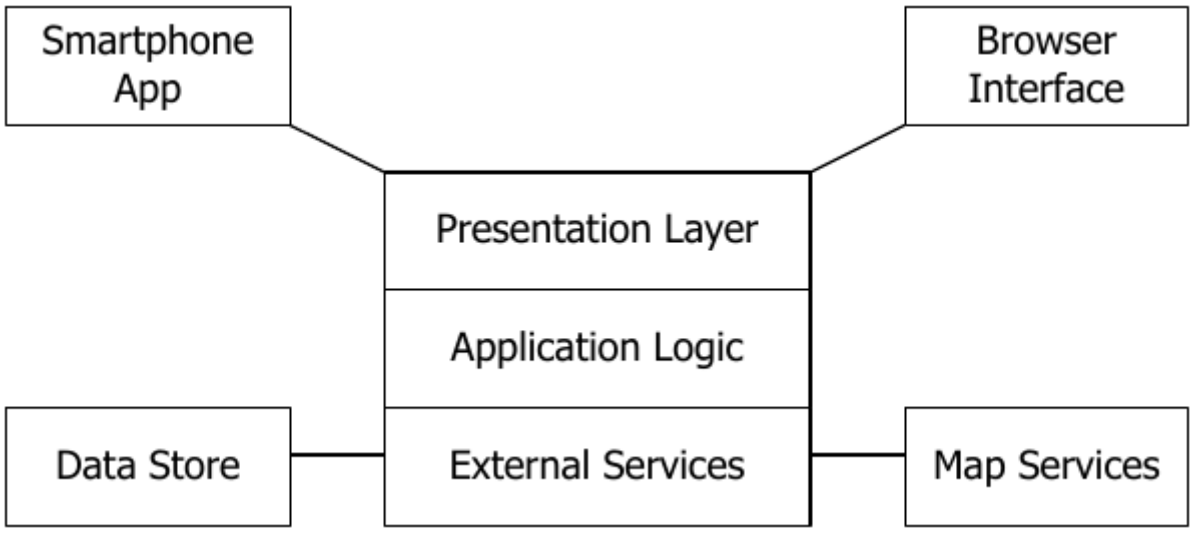
La dicitura "4+1" fa riferimento alle seguenti viste:

- **viste logiche** -> si concentrano sul supporto alle funzionalità e requisiti utente
- **viste di sviluppo** -> si concentrano sui moduli del codice sorgente
- **viste dinamiche** -> si concentrano sugli oggetti e/o i processi a runtime e i flussi di dati
- **viste di deployment** -> si concentrano sulla configurazione, il mapping del software sui server e la distribuzione geografica

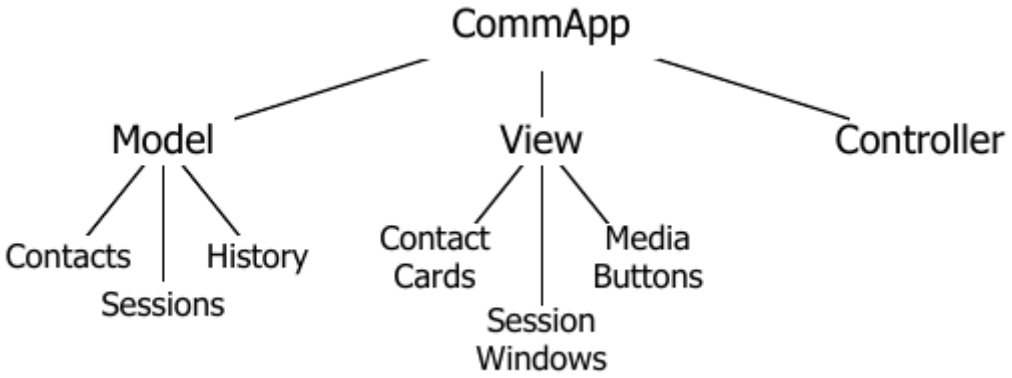


Esempio

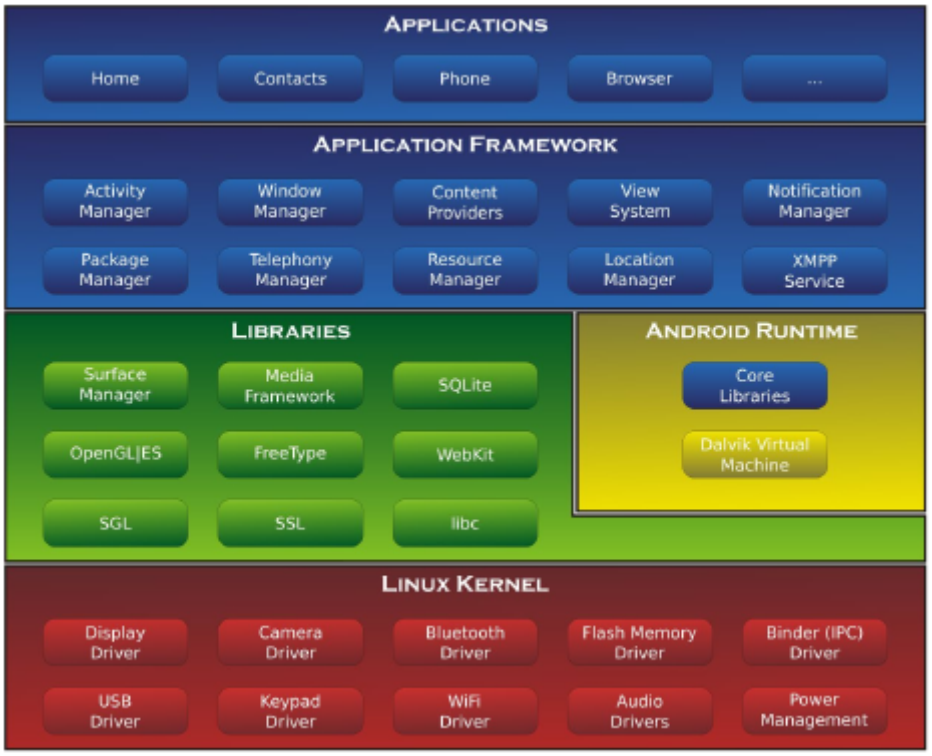
Si consideri un sistema con due interfacce: una per smartphone e una per browser.
Una vista logica del sistema può essere:



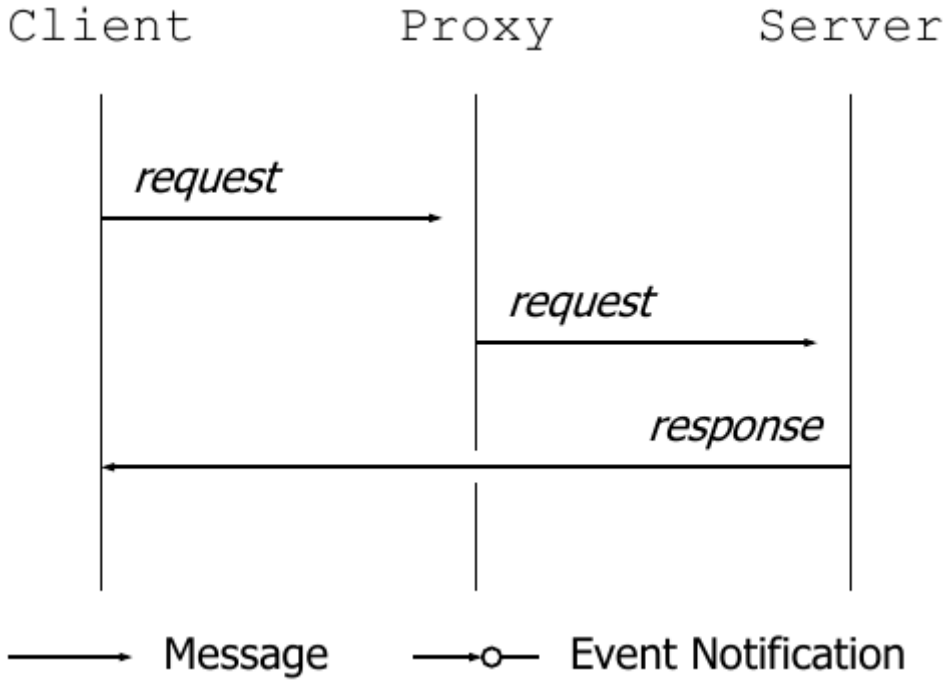
Come vista di sviluppo, si può considerare la gerarchia dei moduli:



o anche lo stack software di Android:



Come vista dinamica invece, si può creare un sequence diagram che mostri le interazioni tra le componenti.



Come trovare un'architettura software

I metodi di progettazione di un'architettura software puntano a derivare l'architettura software dai requisiti funzionali e di qualità.

Ci sono tre principali fattori che guidano la progettazione di un'architettura software: riuso, metodo e intuizione.

- **riuso** -> architettare è un compito difficile che il riuso può aiutare a semplificare. Il fatto che un sistema abbia avuto successo e abbia usato un certo componente, regola o responsabilità favorisce per il riuso di tali soluzioni. Esempi sono gli applicativi bancari e web. La fonte del riuso può essere una versione precedente del sistema, un altro sistema che ne condivide le caratteristiche, pattern architetturali o esperienze a livello di organizzazione. **Domain-specific Software Architecture** e **Product Line Engineering** sono esempi di combinazioni di successo tra architetture software e riuso.
- **metodo** -> ci sono molti linguaggi, modelli di processo e metodi che prescrivono una tecnica sistematica per ridurre le differenze tra architettura software e requisiti.
- **intuizione** -> molti architetti si affidano all'invenzione di elementi di architettura software e le loro relazioni basandosi esclusivamente sulla propria esperienza.

La proporzione tra queste tre fonti di architettura software dipende dall'esperienza di architetti, la loro esperienza formativa e la novità del sistema.

Metodo generale per la progettazione di un'architettura software

Come documentare un'architettura software

Definizioni

Grafo

Si ha un **sistema di interesse**, il quale ha uno o più **stakeholder**. Il sistema ha un'**architettura**, che è descritta da un'appropriata **descrizione**. La descrizione cambia in base a ciò a cui si è interessati: gli stakeholder (*hold* nel senso di avere il potere decisionale) identificano la descrizione.
Gli stakeholder hanno uno o più concern, che consentono di identificare dei punti di vista architetturali. In particolare, il punto di vista è l'insieme di concerti rilevanti per un insieme di stakeholder e che, applicato ad un sistema, dà la vista del sistema. Tra i punti di vista, ne esistono anche alcuni predefiniti. I modelli partecipano a una vista, quindi una vista si compone di almeno un modello. Molto spesso, soprattutto in passato, c'è stata molta attenzione al razionale dell'architettura: perché è fatta in un certo modo? Sarebbe buona pratica, quando si prendono delle decisioni, descrivere i motivi delle decisioni.

Product line engineering

La cilindrata dei motori non cambia al variare della potenza del motore, causa limitazioni della centralina. Ciò è dovuto a un motivo ingegneristico di limitazione dei costi, potendo riutilizzare la stessa centralina per diversi modelli. Anche nell'ingegneria del software, si può applicare il concetto di riutilizzo.

Architettura di riferimento

Nella PLE, non si sviluppa più il codice per un sistema, quanto per un insieme di sistemi. Un'architettura di riferimento è un pattern architetturale predefinito, possibilmente parzialmente o completamente instanziato.

Il riuso sistematico è il riutilizzo predetto delle componenti software, mentre il riuso attivo viene fatto di volta in volta dal caso d'uso.

Tramite una fase di ingegneria della famiglia di prodotti, si sviluppano componenti software per

Scoping

Lo scoping definisce quale funzionalità sono riutilizzabili e quali no. Talvolta, pensare tutti i componenti per il riutilizzo non poterà - prima o poi - ad alcun vantaggio.

Potenziali svantaggi dello sviluppo di componenti riutilizzabili

- **interfaccia complessa**: al fine di
- **rumore dell'organizzazione**: il budget si prende dai prodotti; il family engineering si trova spesso ad affrontare situazioni il team del sistema A deve sviluppare un componente utile anche a B, ma ad un costo maggiore rispetto a quello che sarebbe stato richiesto soltanto da A; in questo caso, occorre trovare un accordo sul budget aggiuntivo.

Stili architetturali

Le applicazioni software, così come le case, hanno il proprio stile. Si hanno le seguenti definizioni:

- **decomposizione in sottosistemi** -> identificazione dei sottosistemi, dei servizi e delle relazioni tra loro
- **stile architetturale** -> pattern per la decomposizione in sottosistemi
- **architettura software** -> istanza di uno stile architetturale
  Nota: un pattern è una soluzione astratta a un problema già esistente.

Applicando uno stile architetturale, si possono identificare i propri sottosistemi. Lo stile architetturale detta la definizione dell'architettura.

Client-server

Uno o più **server** forniscono servizi a istanze di sottosistemi detti **client**. Esiste un'interfaccia dei servizi e spesso i client non conoscono la posizione fisica del server.

Vantaggi: centralizzato, controllo sulle risorse condivise.

Peer-to-peer

È una generalizzazione dell'architettura client-server, in cui i client possono essere server e i server possono essere client.

Esempio: BitTorrent.
Vantaggi: anonimità (seppur parziale), nessun controllo sulle risorse condivise.

Pipes and filters

Un componente legge degli stream di dati come input e produce stream di dati come output.
Si adatta alle applicazioni che richiedono una sequenza di calcoli da eseguire sui dati.
Questo stile architetturale si concentra sulle interazioni dinamiche anziché sulla struttura.

Vantaggi: riuso certificato.

In alcuni contesti, dati i requisiti di alto livello, si procede subito a creare il sequence diagram

Shared data

I dati sono condivisi.

5 - Refactoring

Manutenzione

È più complicato manutenere vecchio codice piuttosto che scriverne di nuovo. Infatti, un codice tocca da più persone - con mindset diversi e in momenti diversi - è più difficile da manutenere. Quando sono presenti tante righe di codice, è più probabile trovare dei bug regressivi.

Si può parlare di manutenzione come gli sviluppatori spendono il loro tempo.

È estremamente importante fare evolvere sistemi di grandi dimensioni.

Refactoring

Il **refactoring** è un'operazione che migliora la struttura interna di un software, senza intaccare il comportamento esterno.
Si tratta di solito di un'operazione di breve durata, che comporta tuttavia un investimenti a lungo termine sulla qualità del sistema.

È importante, oltre a scrivere bene il codice fin da subito, individuare le classi/metodi per cui è utile fare il refactoring.

Motivazioni del refactoring

Ogni parte del codice del sistema ha tre scopi:

1. eseguire la funzionalità -> spesso ci si concentra solo su questo
2. consentire modifiche
3. comunicare correttamente agli sviluppatori che lo leggono -> la complessità è esponenziale

Se il sistema software non fa anche solo una di queste cose, è "rotto". A questo proposito, è utile applicare i pattern in modo da avere un riferimento comune su alcune pratiche del sistema. D'altra parte, si può correre il rischio di sovra-progettare una soluzione, dato che eccedono in manutenibilità, ma non nelle prestazioni. Chiaramente dipende, perché si possono sempre aggiungere più risorse di calcolo al sistema.

Il refactoring consente di migliorare la progettazione del software, rendendolo più facile da comprendere.

Dove applicare il refactoring

- codice duplicato

- routine troppo lunghe
- cicli troppo lunghi o molto innestati
- classi con poca coesione

Come eseguire il refactoring

Processo di refactoring

Alcune best practice nel refactoring sono:

- eseguire piccole modifiche, dato che si potrebbe incombere in errori
- eseguire tutti i test dopo il refactoring, per assicurarsi che il codice ancora funzioni
- se tutto funziona, si passa al refactoring successivo
- se non funziona, si risolve il problema, o si cancella la modifica in modo da avere ancora un sistema funzionante

Martin Fowler

Libri:

- sdfsdf
- fsdsd

Catalogo

Queste sono tutte le possibili attività di refactoring:

Reference. Catalogo.

Esempio 1

Gli statement `switch` sono molto rari nel codice OO progettato a dovere:

- di conseguenza, uno `switch` è un "bad smell" facilmente rintracciabile
- chiaramente, non tutti gli usi degli `switch` sono errati
- non bisognerebbe infatti usare lo `switch` per distinguere diversi tipi di oggetti

Ci sono diversi refactoring in questo caso: il più semplice è la creazione di sottoclassi.

Si consideri il seguente esempio:

```
class Animal {
   final int MAMMAL = 0, BIRD = 1, REPTILE = 2;
   int myKind; // set in constructor
   ...
```

```
    String getSkin() {
        switch (myKind) {
            case MAMMAL: return "hair";
            case BIRD: return "feathers";
            case REPTILE: return "scales";
            default: return “skin";
        }
    }
}
```

i può fare il refactoring il codice nel seguente modo:

```
class Animal {
    String getSkin() { return “skin"; }
}

class Mammal extends Animal {
    String getSkin() { return "hair"; }
}

class Bird extends Animal {
    String getSkin() { return "feathers"; }
}

class Reptile extends Animal {
    String getSkin() { return "scales"; }
}
```

Esempio 2

Quando si lavora con sistemi OO, occorre tenere sempre in considerazione la proprietà di single responsibility di una classe.

Si può dividere una classe in due. Ad esempio, una classe `Customer` che al suo interno ha anche informazioni relativi al numero di telefono. Si può estrarre la classe `Phone`:

```
public class Customer {
    private String name;
    private String workPhoneAreaCode;
    private String workPhoneNumber;
}
```

```
public class Customer {
    private String name;
    private Phone workPhone;
}

public class Phone {
    private String areaCode;
    private String number;
}
```

Nonostante si sia aumentata la dimensione del 50%, non sempre questa metrica è fondamentale: infatti, in questo caso, la dimensione totale è aumentata, ma quella delle singole classi è diminuita. Occorre sempre calibrare le metriche.

Esempio 3

Si estrae un metodo da un altro metodo.

Esempio 4

Si estrae una sottoclasse, magari rendendo la classe originale meno specializzata (ad esempio astratta).

Esempio 5

Si estrae una super classe.

Esempio 6

Si riduce un errore del codice con una Exception.

Esempio 7

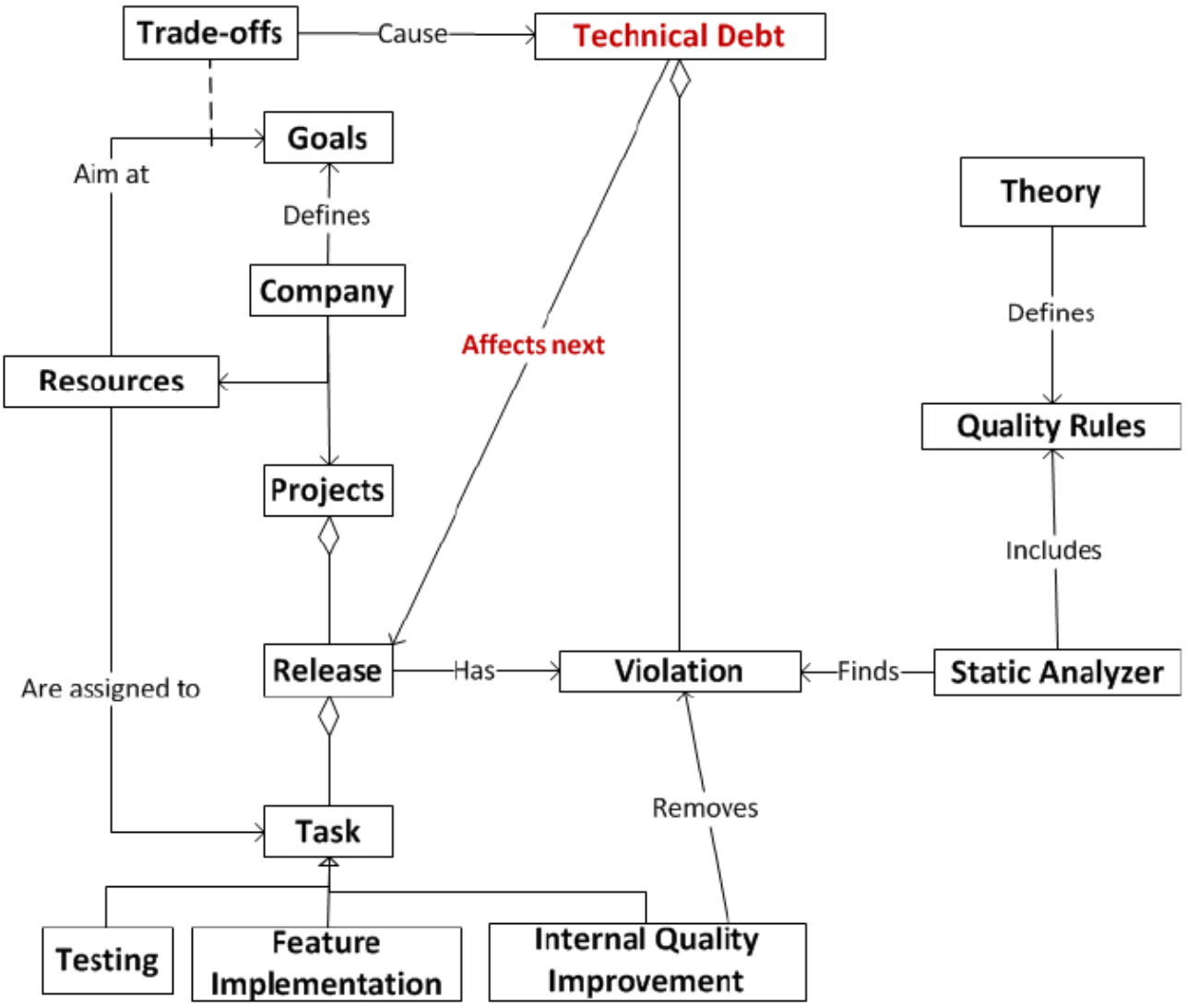
Rimpiazzare un parametro con un metodo esplicito.

Esempio 8

Rimpiazzare una variabile temporanea con una query.

Per le classi, la proporzione tra metodi pubblici e privati può essere un indicatore di quanto sia buggy la classe. Per il progetto, l'attività di risoluzione deve essere sia semplice, sia funzionale: non basta solo definire la metrica per scegliere il metodo su cui applicare il refactoring.

Debito tecnico



Le **teorie** consentono di definire delle regole di qualità (ad esempio, non i possono avere classi con più di 7 attributi). Si sviluppano dunque **analizzatori statici** che verifichino eventuali **violazioni** delle regole di qualità.

Idealmente, avendo **risorse** infinite, non c'è mai bisogno di eseguire refactoring. Infatti, un'azienda di base ha dei task in cui svolgere attività di **testing**, **feature implementation** e **miglioramenti interni della qualità** (ossia refactoring, che rimuovono le violazioni). L'azienda ha diversi goal, spesso in contrasto l'uno con l'altro, quindi occorre calibrare le attività per rientrare nei limiti delle risorse (tra cui il budget). In particolare, lo stato del sistema oggi va a impattare solo la **release** successiva.

6 - Introduzione al machine learning

Definizioni

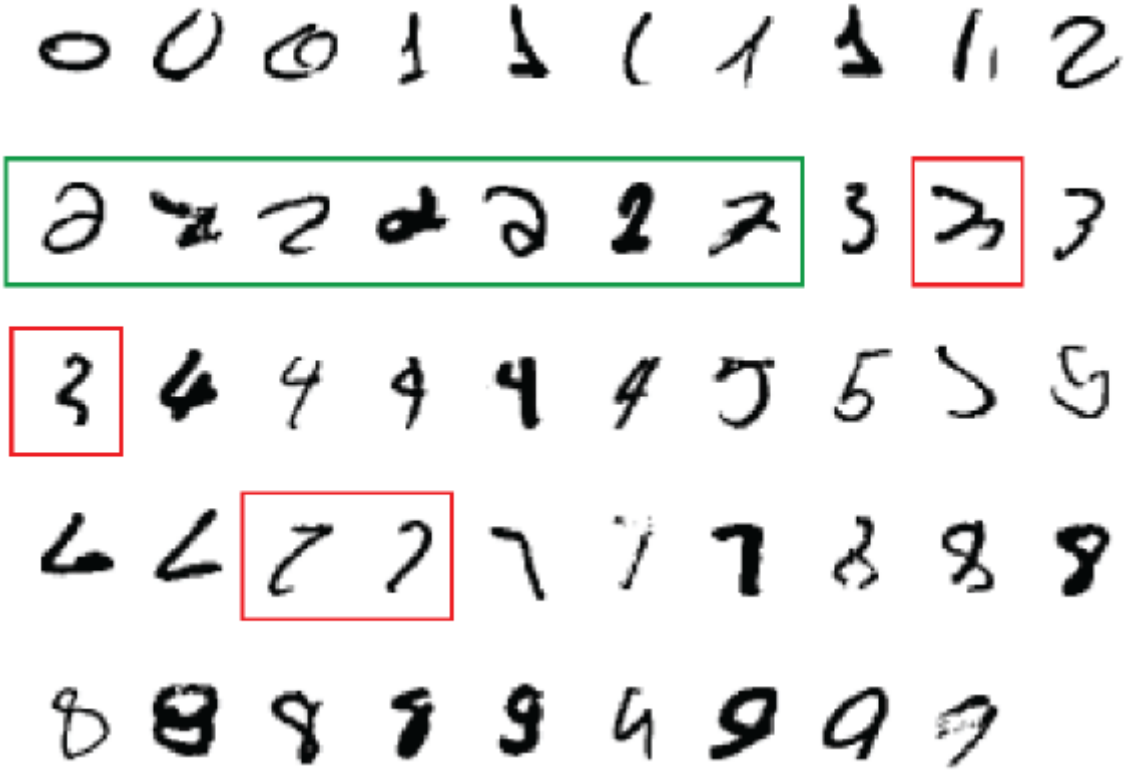
Il **machine learning** è lo studio degli algoritmi che migliorano le loro prestazioni in determinati task dopo aver acquisito certa esperienza.

Si ottimizzano dei criteri di prestazioni usando dati di esempi o esperienza passata.

Il machine learning mette insieme statistica e informatica:

- **statistica** -> inferenza da un campione
- **informatica** -> algoritmi efficienti per risolvere il problema di ottimizzazione e rappresentare e valutare il modello per l'inferenza

Un classico esempio di uso del ML è il riconoscimento di cifre scritte a mano:



Il machine learning punta ad apprendere le associazioni tra delle **feature** X e un **output** Y; in particolare,

Face recognition

Quando si parla di ML, si parla sempre di training set e test set.

Il **training set** è l'insieme di dati di addestramento del modello di ML, che consente di ottimizzare le prestazioni.

Il **test set** è un insieme di dati *non presenti nel test set* che verificano che il modello abbia buone prestazioni anche su dati nuovi che non ha mai visto.

Face database.

Modelli predittivi

Regressione

Uno dei modelli più semplici del ML è la regressione lineare.

Ad esempio, se si vuole predire il prezzo di una macchina, ipotizzando - per semplicità - che l'unica cosa che impatti il prezzo sia il numero di chilometri della macchina.

Siano x un array contenente rappresentazioni numeriche degli attributi della macchina e y il prezzo della stessa. Si può scrivere y in funzione di x come

$$y = g(x|\theta)$$

dove $g()$ rappresenta il modello e θ è l'insieme dei **parametri**.
Nel caso della regressione lineare,

$$y = wx + w_0, \qquad \theta = (w, w_0)$$

I parametri possono essere calcolati in infiniti modi: infatti, se la regressione lineare punta a trovare la retta che riduce maggiormente la distanza tra punti, il numero di modi per calcolare la distanza tra punti è infinito.

Tipologie di apprendimento

Nel ML esistono 4 (principali) tipologie di apprendimento:

- **supervised learning** -> c'è una variabile di interesse che si vuole stimare e un insieme si dati storici sulla variabile
- **unsupervised learning** -> c'è una variabile da analizzare in base alle sue relazioni con altre variabili, ma senza avere i valori della variabile di interesse
- **semi-supervised learning** ->

Terminologia

L'**input** di un modello è un tabella di dati storici che gli vengono dati in pasto. Le colonne rappresentano le **feature** (o **attributi**). Un insieme di valori di più attributi appartenenti alla stessa riga rappresenta un'**istanza**, vale a dire l'esempio indipendente di un concetto da essere appreso dal modello. L'ultimo attributo nella tabella è quello da stimare.

Nel caso di classificazione di metodi come buggy o meno, si possono considerare diversi attributi: data della release in cui è presente il metodo, quanto è usato, quanto usa altri

metodi, il numero di righe di codice e gli smell. Si potrebbe anche considerare il nome del metodo, ma:

- se sono già presenti tutti gli attributi necessari, potrebbe non essere necessario inserirlo
- si può considerare il nome del metodo per tracciare il numero di modifiche fatte

La dimensione di un dataset è data dal prodotto tra il numero di righe e numero di colonne. Tuttavia, non tutte le colonne hanno la stessa qualità, ossia non contribuiscono allo stesso modo alla predizione. Si può considerare la correlazione degli attributi con quello di predizione, ma non è sufficiente, essendo la predizione dovuta a più fattori che agiscono contemporaneamente. Ad esempio, la colonna `esperienza` e `haToccatoIlMetodoInPassato` singolarmente non indicano niente, ma insieme sì.

Oltre il concetto di qualità, è importante il concetto di variazione, ossia che le istanze siano rappresentate lo stesso numero di volte.

Quindi, è vero che *size does matter*, ma non è così semplice valutare la size. Ad esempio, sarebbe inutile duplicare ogni riga del dataset a parità di colonne.

Vedere EPV (entity per value).

Attributi nominali

Output

Ci sono diverse possibilità di rappresentare le informazioni in output:

- tabelle di decisione
- modelli lineari
- alberi decisionali
- cluster

Explainable AI

Per quanto riguarda le regolazioni EU, tutti i sistemi basati su predizioni devono fornire informazioni non solo sulle predizioni, ma anche su come si è arrivati a quella predizione. Ad esempio, GPT e Gemini mostrano il ragionamento all'interno della risposta.

Ciò è dovuto al seguente motivo: si supponga di avere un meccanismo black-box (come le reti neurali, in cui i neuroni non rappresentano niente di concreto e servono solo ad astrarre i dati); d'altra parte, si hanno modelli come quelli di regressione e gli alberi decisionali in cui ogni elemento rappresenta una caratteristica del modello.

Ci si trova spesso di fronte a un compromesso: usare meccanismi precisi, senza sapere perché, e usare meccanismi imprecisi, ma sapere perché.

Weka non accetta variabili di tipo ordinali, né come input né come output. Ad esempio, LOW, MEDIUM e HIGH sono interpretati allo stesso modo di VERDE, GIALLO e ROSSO. Chiaramente, non vanno assegnati valori numerici a LOW, MEDIUM e HIGH.

C'è una relazione lineare tra il numero di bug e il numero di smell in una classe

7 - Introduzione al machine learning e software analytics per support al software engineering

Q&A

Domanda: come trattare i metodi di interfacce che, non avendo un'implementazione di per sé, avranno molti valori di colonne pari a zero?
Risposta: non c'è un vero problema, dato che gli zeri rappresenteranno i valori delle metriche del modello, seppur il modello non sia in grado di comprendere effettivamente il significato che un numero rappresenta.

L'importanza della qualità del software

I bug sono importanti perché hanno rilevanza, sia a livello economico (ref), sia a livello di safety.

Il software si evolve a grandissima velocità, con range da 6 mesi a ogni giorno.

Il desiderio di avere un sistema di grande qualità deriva dal fatto che questo avrà meno possibilità di portare alcuni bug.

I bug possono essere prevenuti in diverse maniere:

- usando strumenti di analisi statica per verificare la qualità del codice
- monitorando il debito tecnico, ad esempio impostando dei quality gate (insieme di regole che devono essere rispettate dal codice affinché il commit sia accettato) con SonarQube/Github Actions.

La manutenibilità è misurata in termini di effort o bug. Tuttavia, il problema dell'effort è che è poco correlabile al concetto di smell (poiché non tiene conto del numero di modifiche che devono essere effettuate), quando invece lo è il bug.

I bug possono essere trovati con:

- unit testing
- CI/CD
- code review

Software analytics

La **software analytics** è una disciplina che combina l'analisi dei dati al software engineering: si occupa di analizzare i dati per migliorare il software. I dati vengono estratti dagli strumenti

ML per i difetti software

Si vuole utilizzare il ML per predire e spiegare i difetti futuri, creando al contempo delle teorie empiriche.

1. Si prendono dati grezzi da Git/Jira
2. Si puliscono i dati
3. SI fa un analisi dei dati
4. Si crea il modello
5. Si può usare il modello per trovare i bug, ma anche utilizzare per spiegare il motivo dei bug (ad esempio quali valori di feature evitare)

Le teorie hanno bisogno di un'osservazione, un'intuizione e una validazione.

Alcune teorie dell'ingegneria del software:

- impatto dei code smell sull'utente
- non avere codice duplicato
- in generale, ogni regola all'interno di SonarCloud
  Le teoria vanno validate o dal punto di vista formale (informatica teorica) o dal punto di vista empirico: prima di facevano degli esperimenti (si facevano sviluppare delle feature su un codice con o senza smell), mentre ora si possono utilizzare le misure storiche.

Misure

Dai Jira e Git si possono estrarre dati (oggetto della prossima lezione). Jira tiene traccia dei ticket, ossia insieme di task assegnati a persone.

Una volta preso il commit di git, si ha anche il riferimento all'id del ticket su Jira.

Linkage: proporzione dei commit che hanno un ticket (non tutti i commit hanno un ticket). Maggiori commit sono senza ticket, maggiori sono le informazioni perse. Di base, si vorrebbe che tutti i commit abbiano un ticket. Ad esempio, se si vuole capire se la maggior parte di commit sia bug fixing o feature implementation, ma molti sono senza ticket, non si può raggiungere una conclusione.

Una volta raccolte le metriche da Git e Jira, si possono identificare i difetti e crearne un database. SI vorrà creare una tabella con uno storico degli injection dei difetti (bug) all'interno del codice dei singoli metodi.

Modello

Si hanno già dei dati di training (da Jira) che vengono dai in pasto ai modelli perché, una volta date delle classi di test, verificano se queste classi sono buggy o meno.

Di base, il modello non classifica in base alla classe, ma fornisce una probabilità che un'istanza appartenga a una classe piuttosto che a un'altra.

Explainability

Dato un modello black box, a seguito dell'addestramento si può tirare fuori una spiegazione. Uno degli approcci è ad esempio tirare fuori un albero decisionale (modello explainable) da una rete neurale (modello non explainable)

CMMI

CMMI: Capability Maturement Model Integration. Misura la maturità di un processo.

I difetti che si vogliono evitare sono quelli in produzione. Si vuole migliorare il profitto per dipendente e il numero di requisiti per release.
Nel grafico, i dati sono stati normalizzati in una scala da 0 a 100: è importante sottolineare il trend a seguito anche delle certificazioni assunte dall'azienda. Ad esempio, dopo ML3 è stato suggerito di usare Jira, che ha permesso di contare i bug in produzione.

Una tecnica utilizzata per il controllo del processo è il processo control graph: serve a garantire la stabilità di un processo. C'è una metrica (ad esempio il numero di difetti, o lo stesso per numero di righe) che non deve variare eccessivamente nel tempo (in questo caso in release), ossia deve rimanere all'interno di un range (upper and lower limit) che vengono calcolati. Quando un dato futuro (ma anche passato) non rientra questi limiti, occorre - come best practice - convocare una riunione per analizzare cosa sia successo.

Release planning: consente di decidere quali release hanno quali ticket.

Cliccando sul grafico (punto verde) dice che al 83% si è confidenti che il numero di bug minori in produzione sarà minore di 5.04.

Github è collegabile a Jira tramite il numero del ticket nel commento del commit.

8 - SZZ - Identificare entità buggy

Principi del ML: size does matter, garbage in, garbage out.

Dal punto di vista del PM, i valori di interesse sono l'effort e il costo dello sviluppo di un codice. Una metrica per determinare se un codice è ben scritto è la presenza di bug o meno.

L'approccio SZZ assume il fatto che tutte le righe di codice modificate all'interno di un commit di tipo bug fix erano tutte buggy.

Per il progetto, dovrà essere creata una tabella (dataset) in cui l'ultima colonna è l'attributo buggy/non-buggy e le altre tutte le feature che comportano la presenza o meno di bug. Tutte le metriche devono essere misurabili. Esempi sono:

- grandezza del metodo
- metodi usati dal metodo
- metodi che usano il metodo
- quante modifiche ha subito il metodo in passato (importante)
- altre misure di complessità
  Nota: se le metriche vengono fatte a release, la metrica sarà la somma delle metriche su ogni commit.

Quindi:

1. si prendono tutti i commit il cui ticket nel commento è di tipo buggy
2. la versione precedente del metodo sarà buggy, quella nuova non-buggy

Problema: si conosce il tipo e la posizione di bug sono a seguito del fix, tramite il commit.

Quando si fa il blame tramite Git (per vedere quando il codice è stato inserito). Potrebbero capitare situazioni in cui si ha (considerando la colonna target come 'il bug è presente' e non 'il bug è stato inserito'):

- il metodo `a()` buggy dalla versione 1.11
- il metodo `b()` buggy dalla versione 1.14
- il metodo `c()` buggy dalla versione 1.16
- tutti i metodi non più buggy dalla versione 1.18, che ha risolto un bug modificando tutti e tre i metodi

Limitazioni

- **backward search impreciso**: diventa difficile capire quanto andare indietro, essendo la riga del dataset un metodo di una release: non si sa quante righe del dataset

corrispondenti a quel metodo devono essere classificate come buggy
    - per risolvere il problema, si possono utilizzare i campi *affect version* e *fix version* di Jira insieme alle date di rilascio delle suddette versioni; se questi due campi non ci sono, si possono stimare.
- **bug regressivi**: si può assumere che il problema si trovava dove sono state effettuate le modifiche
- **refactoring**: il blame potrebbe non indicare esattamente quanto andare indietro, potendo esistere alcuni cambiamenti non significativi (es. cambio di nome di una variabile)
    - soluzione: posto sempre di non poter utilizzare l'affect version, se le modifiche non sono significative, si continua a passare alla versione precedente
- **snoring**: next lesson

Se c'è l'affect version, non si usa il blame e si considerano i metodi interessati come buggy a partire dalla versione indicata. Altrimenti, si usa il blame.

Il processo SZZ si compone di due fasi:

- identificazione delle entità buggy
- identificazione delle release buggy

Per identificare al meglio le versioni, il blame non sempre funziona, ma si può usare l'affected version di Jira.

9 - Snoring

Con l'approccio SZZ, esistono i concetti di:

- injected version (IV) -> versione in cui il bug è stato inserito
- affected versions (AV) -> versioni affette dal bug
- fix version (FV) -> versione immediatamente successiva al commit con il fix del bug
  Deve essere IV <= AV < FV.

Spesso nel ML si ha del rumore dei dati dovuto a bug "dormienti", che quindi provocano **snoring**. Infatti, i bug possono essere scovati solo dopo un'attenta analisi, che può avvenire sì allo stesso momento del'introduzione, ma il più delle volte avviene dopo.

L'approccio SZZ implica che un metodo sia buggy o non buggy a seconda del momento in cui viene effettuato il labeling. Ad esempio, si considerino le seguenti classi e l'evoluzione dei bug nelle varie release (I=Injected, N=Nothing, F=Fixed):

	r1	r2	r3
C1	IF	I	F
C2	I	N	F
C3	II	F	F

Dato che la fix permette di capire le classi difettose o meno, a seconda di quando si effettua la misurazione, la difettosità cambia:

- se ci misura alla release 2

Classes	r1
C1	ND
C2	ND
C3	D

- se si misura alla release 3

Classes	r1	r2
C1	ND	D
C2	D	D
C3	D	D

  Nota: non importa a che punto della release di esegue la misurazione.

Senza la fix non si conosce neanche il metodo, mentre l'affected version server a capire quanto andare indietro nel tempo. In pratica, si può conoscere la presenza di un bug nel passato solo a seguito di una fix futura. È un discorso analogo alla presenta di un testimone in un processo, o la dimostrazione di una teoria errata.

Tuttavia, se un difetto è presente e non si ha la sua fix, ci sarà sicuramente un rumore nei dati.

I dati e le loro relazioni cambiano nel tempo. Si vorrebbero:

- dati freschi
- tanti dati (che non potranno essere tutti freschi)
  e c'è il problema dello snoring.

Positive & negative

Il **true** è una misura di predizione. Il valore **actual** è il valore vero di un'istanza, mentre il valore **predicted** è quello predetto dal modello.

In un contesto specifico, il **positive** indica ciò che si cerca: è una convinzione, su cui occorre essere d'accordo. In particolare, **positive** o **negative** indica il valore stimato. D'altra parte, **true** e **false** indicano la correttezza della predizione. Si hanno allora le seguenti combinazioni:

- **true positive** è un'actual positive/predicted positive
- **false positive** è un'actual negative/predicted positive
- **true negative** è un'actual negative/predicted negative
- **false negative** è un'actual positive/predicted negative

Questo approccio viene solitamente applicato sulla predizione, ma nel caso dello snoring si applica sulla misurazione, essendo questa - potenzialmente - errata a seconda del momento in cui viene eseguita. Lo snoring può dare vita al false negative, ma non al false positive.

Domande

Introduzione

- Quanto dorme un difetto?
- Quanto una classe (o entità) "russa"?
  - "Non sempre chi dorme russa" ~ D.F.
  - un bug russa se causa una labeling incorretta per quella data release; ne basta trovare uno per l'entità considerata, dato che gli altri non russano. Infatti, un bug russa per una release a seguito di una misurazione effettuata in un'altra release. Dunque, un'entità per non russare ha bisogno di un altro bug che viene fixato nella stessa entità.

In generale, ci si aspetta che l'avanzare del tempo migliori l'efficienza della predizione, dato che l'applicazione verrà utilizzata di più.

Ci interessa non tanto che ci sia snoring, quanto che esso impatti o meno sulla predizione.

Quando si eliminano le release dal dataset, non si eliminano tutte, ma si mantiene la porzione buggy, dato che lo snoring si nasconde solo dietro le istanze classificate come negative, producendo false negative. Il numero di release rimosse può differire il base al progetto.

10 - Predicting bug-inducing tickets

Introduzione

La **bug prediction** è una branca della ricerca che si occupa di identificare i bug in progetti software prima che si presentino.

Fin'ora la bug prediction si è occupata di cercare i bug - già presenti, ma non ancora trovati - all'interno di classi, metodi ecc.

Lo studio punta a fare predizioni sulla possibile presenza dei bug a partire dai ticket che rappresentano un requisito. In particolare, si definiscono, misurano e valutano 62 feature per un nuovo task detto **Ticket-Level Prediction** (**TLP**).

In un processo di sviluppo software, un **ticket** può avere tre diversi stati:

- **open** -> il ticket è stato appena aperto e indica una nuova funzionalità da sviluppare, un bug da risolvere o una funzione da ottimizzare
- **in progress** -> il ticket è in lavorazione; si possono definire altri due sotto-stati:
  - **assigned** -> il ticket è stato assegnato a uno sviluppatore
  - **first-commit** -> lo sviluppatore ha iniziato a lavorare sul ticket e ha eseguito il suo primo commit, al quale possono seguirne altri
- **closed** -> il ticket viene chiuso dopo il completamento della feature

A un ticket sono collegati uno o più commit, i quali contengono all'interno del loro commento l'id del ticket.

Il commit è quello che potrebbe effettivamente contenere il bug, mentre il ticket raccoglie più ticket correlati. In questo caso, si guarda soltanto al ticket più che al commit che contiene il bug effettivo. Questa scelta è dovuta al fatto che prima una predizione viene fatta, meglio è. Con questo in mente, si vuole adoperare la bug prediction per adoperarsi per tempo alla risoluzione/prevenzione del bug. Tuttavia, è un'arma a doppio taglio, poiché maggiore è l'anticipazione e minore è l'accuratezza della predizione.

Dato un ticket, dunque, si prendono la data di assegnazione e quelle dei vari commit e ci si pone in tre istanti temporali:

- appena prima dell'assegnazione
- appena prima del primo commit
- dopo l'ultimo commit
  Si vuole valutare l'accuratezza delle predizioni nei tre istanti di tempo.

Design

Ci si pone due domande:

- **RQ1**: la prossimità temporale impatta l'accuratezza del TLP?
- **RQ2**: la prossimità temporale impatta il potere predittivo delle feature TLP?

La procedura di misurazione è fatta nel seguente modo. Ci si pone nei panni del PM e si cercano dataset JIT già presenti e riutilizzabili, dai quali si ricavano dei dataset TLP: se almeno uno dei commit associati a un ticket è buggy, allora lo è anche il ticket. Il dataset è suddiviso in training e test set e la valutazione è fatta sull'accuratezza e

Notare che le feature dei ticket non esistevano prima ed è stato necessario progettarle. In particolare, sono state definite 7 famiglie:

- codice -> relativi alla codebase (non quella che implementa il ticket, ma il restante), indica quanto è semplice maneggiarlo, il debito tecnico ecc.
- sviluppatore -> relativi allo sviluppatore a cui è assegnato il ticket, tra cui la familiarità dello sviluppatore con il progetto.
- temperatura esterna -> quante volte il ticket è soggetto a cambiamenti a causa di azioni provenienti dall'esterno (ad esempio nuove leggi che implicano la modifica dei ticket); ad esempio la località temporale, vale a dire quanti ticket buggy in una finestra temporale (misurata in ticket) che termina sul ticket stesso.
- temperatura interna -> quante volte il ticket è soggetto a cambiamenti dovuti agli stessi sviluppatori (modifica della descrizione, collegamento a un altro ticket ecc.); ad esempio il numero di commenti sul ticket
- intrinseco -> relativi al ticket stesso, ad esempio la priorità o il tipo (bug, improvement, new feature, subtask)
- requisito vs requisito -> relative a un ticket rispetto a un altro ticket; in particolare, si definiscono delle misure di distanza tra i ticket, come la distanza di Levenshtein minima (o simiglianza massima) tra i titoli dei ticket.
- JIT -> feature dei dataset sui commit già visti; ad esempio, il numero di LOC aggiunte o il numero di commit
  Queste 62 feature totali vanno misurate per ogni ticket e cambiano a seconda del momento in cui vengono misurate.

La misurazione delle feature avviene nel seguente modo. A partire dai dataset JIT, si estraggono le colonne di buggyness e le coordinate (dove si trova il bug).

Tra tutti i progetti, ne sono stati scelti solo due (HIVE e HBASE), poiché sono quelli con percentuale di buggy commit associati ai ticket maggiore, oltre ad avere meno rumore nei dati. In totale, si hanno 11000 ticket.

RQ1

Variabile indipendenti: istante di misurazione
Variabili dipendenti: metriche di accuratezza

H10: la prossimità temporale non influenza l'accuratezza della predizione

RQ2

Variabile indipendenti: istante di misurazione
Variabili dipendenti: IGR
H10: il potere predittivo delle feature TLP non dipende dall'istante di misurazione

Tecnica di validazione

Si applica un approccio sliding time window: per ogni finestra, si prende

Si tiene anche conto del fatto che può esserci context drift: le relazioni tra le feature sulla buggyness possono cambiare nel tempo. L'approccio a finestra scorrevole ne riduce l'impatto.

Risultati

RQ1

Per la metrica di AUC, si ha una maggiore accuratezza sulla predizione al passare del tempo.

Per la metrica Kappa (quanto si va meglio o peggio rispetto a un predittore dummy, ossia non addestrato sui dati), si può dire qualcosa sul bug che verrà fuori solo guardando il ticket.

RQ2

Le stesse feature, misurate in momenti diversi, hanno un'importanza diversa sul risultato finale. Segue che non c'è una famiglia di feature ottima, così come non vi è un momento più importante.

Infatti, occorre prendere le coppie (famiglia di feature, istante di misurazione) migliore. Alcune delle più significative sono:

- numero di partecipanti e numero di commit in parallelo quando lo stato è open
- il numero di attività (sul ticket) quando in progress
- JIT e il numero di linguaggi quando in closed

12 - Evaluation and metrics

Valutazione di un classificatore

In qualsiasi sistema informatico che include una componente di ML, questa deve essere validata oltre che configurata. Occorre quindi stabilire quanto un classificatore è accurato.

Perché? Perché il futuro è simile al passato; l'idea è dunque che non ci interessa capire quanto il classificatore è accurato sui dati che già si hanno, piuttosto quanto lo sarà su dati nuovi.

Training e testing

Ci si concentra su classificatori binari. Si ha sempre un valore *actual*, ossia quello reale del'istanza, e uno *predicted*, predetto dal classificatore.

Dataset

In generale, per capire quanto è accurato un classificatore, è necessario avere a disposizione un dataset.

Esistono diverse tecniche di validazione dei classificatori. In generale, una tecnica migliore indica il classificatore migliore, ma dipende sempre da caso a caso. Ergo, non esiste una metrica sempre migliore.

Stima holdout

Il metodo **holdout** consiste nella separazione del dataset in training set e test set.

Il training test serve a calibrare il modello, mentre il test set permette di valutare quanto il modello sia affidabile su dati nuovi.

Il problema di questa tecnica è che i campioni dei set potrebbero non essere rappresentativi della realtà, altrimenti il modello non potrà fare predizioni accurate. Di conseguenza, si vogliono avere dei set rappresentativi, sia nel training che nel test.

Un altro problema è la proporzione della variabile di interesse sulla quale di fanno predizioni: è necessario che training set e test set rappresentino questa variabile in maniera proporzionata. Si possono quindi avere i due set **stratificati**: training set e test set avranno la stessa proporzione delle istanze della variabile di interesse. In realtà, questa tecnica dovrebbe essere applicata su tutte le variabili, ma questo non è possibile per tutte.

Metodo repeated holdout

Se i dati non sono ordinati e il tempo non ha impatto, si può ripetere la stessa tecnica di holdout su diversi sotto-campioni del dataset. In particolare, si vorrebbe anche che tutti i campioni venissero usati lo stesso numero di volte sia nel training che nel test set.

C'è un punto di incontro tra il costo della valutazione e l'accuratezza del modello finale.

K-fold cross-validation

K-fold cross-validation è una tecnica di validazione che evita la sovrapposizione dei test set, in modo che tutte le parti saranno usate nel test set lo stesso numero di volte.

1. si divide il dataset in K sottoinsieme di egual misura
2. si usa ogni sottoinsieme alla volta per il testing e il restante per il training
   Ciò implica che l'algoritmo di addestramento è applicato a k training set diversi.

Per aumentare la randomness dei dati, si può applicare questa tecnica più volte, assegnando i dati a parti ("foglie") diverse.

In generale, per ogni coppia training-test set si ha poi un valore di accuratezza, dei quali si calcola la media. Si può allora definire il modello migliore, dunque si riaddestra il modello da zero. Il k-fold viene ogni volta applicato sul modello non addestrato.

La pratica e la ricerca dicono che k-fold va applicato 10 volte.

Leave-one-out cross-validation

Pro: tutti
Contro: costo computazionale

Si toglie una sola riga alla volta per il test set, rendendo il dataset più simile possibile a quello di produzione.

Bootstrap

In generale, i dati del test set non devono essere presenti nel training set.

Tuttavia, la tecnica **bootstrap** consente di costruire un training set utilizzando un campionamento dei dati dal dataset con rimpiazzo, mentre il test test è composto dalle istanze rimanenti. Dato che il training set potrebbe contenere lo stesso dato ripetuto più volte, oppure non presentare per nulla un certo dato, comporta diversi rischi, come quello di overfitting.

Tecniche di validazione per serie temporali

Spesso è necessario mantenere l'ordine dei dati per la creazione di training e test set.

Ordered holdout

Come holdout, ma il training è composto dalle prime istanze e il test dalle ultime.

Walk-forward

Il walk-forward consente di dividere il dataset in parti uguali, che vengono poi ordinate temporalmente. Ad ogni esecuzione, per il training set vengono utilizzati tutti i dati precedenti alla parte considerata, mentre essa è usata per il test set.

Concept drift: i dati lontani temporalmente non sono così realistici rispetto ai dati più vicini temporalmente. Dunque, alcuni dei dati passati devono essere rimossi, ma occorre capire quanti.

Sliding window

Per risolvere questo problema, si può utilizzare un approccio sliding window, in cui ti tiene conto solo delle ultime N unità del dataset per il training. Il problema principale è la dimensione della finestra: troppo piccola implica pochi dati, troppo grande non risolve il problema di prima.

Nota paper: le tecniche che mantengono l'ordine dei dati hanno una maggiore accuratezza delle altre.

Metriche di prestazioni

Il concetto di accuratezza è definito in base al dato considerato come positivo. La definizione di un positivo è predisposta dal dominio.

In generale, un falso negativo ha costi maggiori rispetto a un falso positivo.

Si hanno le seguenti metriche:

- **recall**: $\frac{TP}{TP+FN}$ -> la proporzione di istanze positive classificate come positive sul numero totale di istanze realmente positive (tra cui quelle classificate erroneamente come negative)
  - indica quante volte ci si prende sul totale
  - problema: non basta, perché basta classificare tutte le istanze come positive per ottenere una recall alta
- **precision**: $\frac{TP}{TP+FP}$ -> la proporzione di istanze positive classificate come positive sul numero di istanze classificate come positive ("a lupo a lupo")
  - indica quante volte la predizione di positive era corretta
  - in generale, a parità di condizioni, si preferisce avere una recall alta piuttosto che una precision alta, dato che un falso negativo costa di più di un falso positivo
  - problema: si ottiene facilmente classificando tutto come negativo
- **F1-score**: media armonica tra precision e recall
- **accuratezza**: $\frac{Tx}{Tx+Fx}$ -> in realtà non implica niente

- **Kappa**: indica l'accuratezza rispetto a un classificatore dummy (random)
    - misurare il Kappa (deve essere in teoria tra 0.2 e 0.4)

Curve ROC

Precision e recall sono affette dal problema della threshold.

Si calcola una curva che calcola precision e recall al variare di ogni threshold e poi si calcola l'area sotto la curva (AUC) che indica il valore dell'accuratezza.

Problema: non tutte le threshold valgono ugualmente, dato che ognuna ha un significato diverso. L'idea dovrebbe dunque essere definire un insieme di threshold che abbia senso, attribuire a ognuna un peso e poi fare un media pesata.

13 - Feature selection

L'attività di **labeling** consiste nell'attribuire ad un metodo di una release il fatto che sia stato buggy o non buggy. Similmente, durante la fase di **predizione**, si predice se un metodo in una certa release è stato buggy o meno.

Una **feature** è una caratteristica delle istanze del dataset utile al predittore ad eseguire predizioni. In particolare, le colonne rappresentano le feature. Le feature devono essere **correlate** alla variabile di interesse, in modo da rendere la predizione più precisa possibile.

Dunque, dato un dataset, si può eseguire una **selezione delle feature**. Questa attività ha diverse motivazioni:

- riduzione del costo: ogni feature data in pasto al modello implica che questo faccia fitting su di essa, dunque il costo di addestramento aumenta per ogni feature presente nel dataset
- si hanno prestazioni migliori per quanto riguarda l'apprendimento rispetto all'utilizzo dell'intero insieme di attributi
- si diminuisce il costo di aggiornamento del dataset, dato che il numero di colonne impatta il costo delle misurazioni
  Inoltre, ovviamente, a parità di prestazioni, è meglio un modello che utilizza meno feature piuttosto che uno che ne utilizza di più - anche indipendentemente dai costi.

Un modello di predizione deve eseguire predizioni corrette ed essere facilmente utilizzabile. A parità di prestazioni, si preferisce dunque un modello meno costoso e potenzialmente esplicabile.

Eliminando alcune feature, il modello potrebbe migliorare in termini di prestazioni per diversi motivi:

- ridondanza
- incorrettezza della feature (rispetto alla variabile di interesse)

Metodi di feature selection

Due approcci:

- filtri
- wrapper

Filtri

Si utilizzano concetti matematici (come information gain o spearman correlation) per misurare quanto le feature impattano la variabile di interesse.

Il problema del ranking non tiene conto delle correlazioni interne tra le feature, ma solo di come queste sono correlate alla variabile di interesse. Inoltre, non si può sapere quante feature scegliere tra le prime, non avendo una misura dell'impatto sull'accuratezza. Spesso, c'è un compromesso tra la rimozione delle colonne e il mantenimento. Idealmente, si vorrebbero delle feature molto correlate con la variabile di interesse, ma non tra loro.

Metrica EPV (Entity Per Value): valore minimo dell'occorrenza per ogni valore per ogni variabile.

Wrapper

Per quanto l'approccio filtro sia veloce, non dà informazioni sulla correttezza della selezione delle feature.

L'approccio **wrapper** consiste nel dividere il dataset in tre parti: training, validation e test.

Approccio esaustivo

Si provano tutte le combinazioni di feature in tutte le loro cardinalità. "Provare" significa addestrare un modello su una data combinazione di feature e verificarne l'accuratezza. L'approccio include la selezione di un predittore e anche di una metrica di correttezza in base alla quale prendere decisioni.

Lo svantaggio di un approccio esaustivo non è che costa tanto, ma che costa troppo, dipendendo dal numero di feature. Inoltre, questo approccio potrebbe rimuovere feature importanti o non rimuoverne di meno importanti, dato che le scelte vengono eseguite solo sul test set.TU

Approcci greedy

Questo tipo di approcci non trovano la combinazione perfetta, ma la combinazione perfetta rispetto a una metrica di accuratezza.

Il feature set trovato

Backward search

Si va a ritroso:

- si prendono inizialmente in considerazione tutte le feature
- si rimuove una feature alla volta
- ci si ferma quando non si vede un miglioramento nei risultati

Esempio:

- si considerino 4 feature A, B, C, D
- si inizia con cardinalità 4: ABCD
    - si misura il valore dell'accuratezza
- si passa a cardinalità 3: ABC, ABC, ACD, BCD
    - se tutte queste restituiscano un valore minore di ABCD, si sceglie quest'ultima
    - se, ad esempio, il valore di ABC è maggiore o uguale di quello di ABCD

Forward search

Si inizia con la cardinalità minore e si aggiunge di volta in volta una feature, scegliendo sempre la combinazione migliore (e con cardinalità più bassa) e fermandosi quando se ne trova una peggiore.

Ci si aspetta - ma non è detto che sia così - che il numero di feature selezionate da forward search sia minore di quelle scelte da backward search. In generale, l'approccio razionale è applicare entrambe le tecniche, ma dipende tutto dal contesto. Se, ad esempio, si da che in produzione si possono utilizzare al massimo 10 feature, si può applicare backward search a partire da combinazioni di 10 feature.

Errore comune nel feature selection: utilizzare l'intero dataset al posto del solo training set. Utilizzare il tab classify, che utilizza meta-modelli al posto dei modelli veri e propri.

14 - Weka

Weka non riconosce le scale ordinali e le tratta come nominali, come (HIGH, MEDIUM, LOW). Per risolvere il problema:

- si può rimuovere la colonna con la scala ordinale
- si scompone la colonna in n colonne, una per valore, tutte true/false (meglio non mettere 0/1, perché si assegnerebbe un valore numerico a ognuna)
- se nella scala vi è un valore particolarmente interessante, si potrebbero usare solo due colonne, una per il valore e una per il non-valore, o addirittura una sola colonna true/false per indicare il valore d'interesse (ad esempio HIGH)

Weka non gestisce il nome del file, ma il nome del dataset è specificato nella prima riga del file `.arff`, dopo `@relation`. Il tag `@attribute` indica il nome della feature e il tipo, che può essere:

- `numeric`
- `date`
- label, indicata tra parantesi graffe (ad esempio `{true,false}`)

Il tag `@data` indica l'inizio del CSV contenente i dati. È possibile aprire un CSV direttamente da Weka.

La metrica areaUnderTheROC prende in considerazione tutte le threshold in maniera equa. D'altra parte, la metrica F1 si basa su un'unica threshold, quindi varia al variare di quest'ultima. In entrambi i casi, ci sono pro e contro. L'ideale sarebbe avere un peso per ogni threshold.

15 - Accuratezza dei classificatore

Si vuole comparare l'accuratezza (precision/recall/AUC/Kappa) di tre classificatori (RandomForest/Naive Bayes/Ibk) sui progetti assegnati, utilizzando la tecnica di validazione k-fold cross validation, con $k = 10$.

Quando si fanno i grafici dei risultati, si considera come classificatore migliore quello che ha la mediana più alta per la misura di accuratezza considerata.

16 - What if I had no smells?

Il concetto del paper è quantificare il numero di difetti quando non sono presenti smell.

- **failure/errore**: comportamento inatteso tangibile del sistema
- **bug/difetto**: porzione di codice che, se eseguita sotto determinate circostanze, produce una failure
- **smell**: porzione di codice che non impatta la qualità esterna del sistema, ma che potrebbe essere scritta meglio
  Classi con più alta densità di violazioni presentano una maggiore defect injection frequency, ossia frequenza di difetti inseriti all'interno del codice per linea di codice toccata.

Facendo un'analisi what-if, si può stimare cosa sarebbe accaduto se il codice non avesse riportato code smell.

Passi

1 - Seleziona delle metriche

Vengono definite una serie di metriche - che poi diventano feature - sul codice in questione. Esempi sono:

- LOC
- NFix (numero di bug risolti)
- NAuth (numero di autori)
- NSmells (smell presenti)

Una volta definite, le metriche devono essere misurate. Questo può avvenire anche in istanti temporali diversi, dando vita a più versioni della stessa metrica.

2 - Manipolazione del dataset

Si supponga che A sia il dataset originale (supponendo anche di aver mantenuto solo il 20% più vecchio - quindi più affidabile - del dataset). Si definiscono:

- B+: porzione del dataset con un numero positivo di smell
- C: porzione di A senza smell (complemento di B+)
- B: deriva dalla manipolazione di B+, in cui il numero di smell è stato posto a zero

Ci si vuole chiedere, avendo osservato B+, qual'è la differenza con B predetto.

Si può osservare il valore (in media) delle feature in ogni porzione del dataset nella seguente tabella:

	Mean			Correlation (ρ)		MW
Variable	**A**	**B**	**C**	**NSmells**	**Defectiveness**	**A vs. B**
Size	289	271	303	0.31*	0.18*	0.027
LOC_touched	57	49	60	-0.01	0.25*	0.894
NR	9	8	10	0.02	0.38*	0.916
NFix	3	2	3	0.02	0.51*	0.515
NAuth	0.7	0.7	0.7	-0.07	0.24*	0.574
LOC_added	31	21	39	-0.04	0.18*	0.926
MAX_LOC_added	21	15	26	-0.04	0.18*	0.922
AVG_ LOC_added	12	9	14	-0.03	0.14*	0.796
Churn	16	14	17	0.01	0.12*	0.523
MAX_Churn	10	9	11	0.02	0.10*	0.476
AVG_Churn	7	7	6	0.02	0.09*	0.421
ChgSetSize	6	6	6	-0.06	0.29*	0.723
MAX_ChgSet	3	3	3	-0.05	0.19*	0.765
AVG_ChgSet	2	2	2	-0.04	0.06	0.909
Age	40	42	38	0.18*	0.02	0.084
WeightedAge	11	13	10	0.06	0.10*	0.223
NSmell	12	0	0	-	0.10*	-

Per ogni variabile, si misura anche la correlazione con il numero di smell e la difettività. Il ρ-value indicato quantifica la correlazione.

3 - Selezione del modello

Dato il dataset A, si scelgono 4 classificatori (Begging, BayerNet, J48 e Logistic) e si utilizza la tecnica di cross-validation leave-one-out. Le misure considerate sono precision e recall.

Il modello migliore risulta essere bagging.

4 - Risultati

Si applica il modello di Bagging sui dataset A, B+, B e C, ossia si usano questi come test set. Nello specifico, i dataset A, B+ e C sono già stati visti in fase di addestramento, mentre B no, avendo una colonna modificata. I primi tre sono tuttavia utili a comprendere meglio i risultati su B.

Per costruzione, si ha che 69 difetti si 135 (circa la metà) non sono dovuti a smell. Tuttavia, i restanti 66 sono potenzialmente dovuti alla presenza di smell.

Di questi 66, se si rimuovono gli smell, si avrebbero avuto solo 38 difetti. Quindi, 66-38=28 sono i difetti potenzialmente causati da smell, vale a dire circa il 20% del totale.

Dataset A		Dataset B+		Dataset B	Dataset C	
A	E	A	E	E	A	E
135	120	66	57	38	69	63

Validità dei risultati

A e B sono correlati positivamente se, se si osserva un aumento di A, si osserva anche un aumento di B e viceversa. La causalità indica che l'aumentare di A implica anche l'aumentare di B, ma questo non è sempre deducibile. Esiste anche la correlazione negativa, per cui all'aumentare di una A si osserva una diminuzione di B. L'azionabilità (actionability) indica che una variabile può essere modificata. Ad esempio:

- il numero di righe tocca in una fix non è actionable, dato che non si può dire a uno sviluppatore di modificare poche righe di codice
- il numero di smell introdotti è actionable, dato che si può limitare imponendo dei paletti

Progetto

Fare una rivisitazione del paper per i progetti assegnati. In particolare:

1. creare A (milestone 1)
2. confrontare l'accuratezza di 3 classificatori su A (milestone 2)
3. scegliere il classificatore migliore
4. calcolare la correlazione di ogni feature con bugginess (con Spearman)
5. trovare la feature azionabile con la correlazione più alta con la bugginess (ad esempio il numero di smell), ossia AFeature
6. prendere dall'ultima release di A il metodo con maggior numero di smell (in generale, quello con più alto valore di AFeature), ossia AFMethod; questo è il metodo con beneficio atteso più alto nella rimozione di AFeature
7. modificare AFMethod e creare AFMethod2 tramite refactoring, diminuendo AFeature (portandola ragionevolmente a zero, anche se non sempre è possibile)
8. si ricalcolano tutte le feature su quel metodo, ossia AFeature2
9. confrontare le feature originali relative ad AFMethod con queste ultime

Note:

- il numero di autori aumenta quando si fa il refactoring
- il numero di release o commit può non variare, dato che si potrebbe dire "avrei potuto fare la stessa versione in quest'altro modo"

Domande da porsi:

- in AFMethod2, ci sono feature positivamente correlate con la bugginess che sono aumentate?
    - se sì, magari non è stata migliorata la manutenibilità in AFMethod2 rispetto a AFMethod
- stessa domanda, ma con correlazione negativa
    - se sì, magari è stata migliorata la manutenibilità

10. Creare:
    1. La porzione B+
    2. La porzione C
    3. La porzione B
11. addestrare il classificatore scelto su A, ossia BClassifierA
12. Predire A, B, B+, C e creare una tabella come nel paper (table 5)
13. analizzare la tabella e rispondere alla domanda: quanti metodi buggy potrebbero essere stati prevenuti avendo zero smell?
    1. in tutto
    2. in proporzione
    3. al di fuori da quelli prevenibili

Fare considerazioni sui progetti, facendo anche diverse prove.